

Minding the Retrieval Gap in RAG: An Oracle vs. Embedding RAG Analysis on Multilingual Regional QA

Younes Bouhmidi

Department of Informatics

University of Oslo

younesb@uio.no

Abstract

Open-ended regional question answering asks for locally grounded facts, such as the garden associated with a Czech town or the genre of a Slovak music association, that broad multilingual benchmarks fail to capture and current large language models (LLMs) struggle to answer. We study how retrieval-augmented generation (RAG) can close this gap on CUS-QA, a dataset of native-speaker questions in Czech, Slovak, and Ukrainian. To separate retrieval from generation errors, we compare Oracle RAG, fed the known source page, against Embedding RAG, which retrieves same language chunks via a dense multilingual embedding model. We evaluate four open weight generators, Tiny Aya Global (3.35B), Tiny Aya Water (3.35B), Llama 3.1 8B, and Qwen3-30B-A3B, with LLM-as-a-judge M-Prometheus and chrF on the hidden test. Retrieval is the dominant bottleneck: replacing exact page context with realistically retrieved chunks drops answer correctness from 0.953 to 0.890 even for the strongest model. Qwen3-30B-A3B delivers the best practical performance, and a top- k ablation shows that larger models benefit from more retrieved context while smaller Tiny Aya models are distracted by it.

1 Introduction

Open-ended regional question answering (QA) tests a form of knowledge that is easy to miss in broad multilingual benchmarks, where tasks often emphasize multiple-choice formats or globally known facts. A question may ask for the garden associated with a Czech town, the genre of a Slovak music association, or the name of a Ukrainian local institution. The correct answer is often a short named entity, but it may appear in an inflected form, with local spelling conventions, or inside a page whose relevance is obvious to a native speaker but opaque to a generic LLM. CUS-QA (Libovický

et al., 2026) was introduced to target this setting through native-speaker written questions grounded in Czech, Slovak, and Ukrainian Wikipedia pages.

Retrieval-augmented generation (RAG; Lewis et al., 2020) is a natural fit for CUS-QA because the questions are tied to Wikipedia evidence. Yet a RAG system can fail at two different points. The retriever may not find the right source page, or the generator may fail to extract the answer even when the evidence is present. These two failures have different practical implications: improving the generator cannot repair a missing document, while improving retrieval may not help if the generator is easily distracted by extra context.

This leads to four questions that guide the experiments:

1. How large is the gap between exact-source Oracle RAG and realistic Embedding RAG?
2. Does increasing retrieval depth from top-5 to top-10 help, or does it introduce distracting context?
3. Which open-weight generator gives the best practical quality under runtime and memory constraints?
4. Do semantic LLM judging and lexical chrF evaluation lead to the same conclusions?

We make three empirical contributions. First, we implement both Oracle and Embedding RAG for the native-text CUS-QA track. Second, we compare four open-weight generators: Tiny Aya Global (3.35B), Tiny Aya Water (3.35B), Llama 3.1 8B, and Qwen3-30B-A3B. Third, we analyze retrieval coverage, top- k effects, development-set M-Prometheus accuracy, hidden-test Codabench¹ chrF, and practical quality-efficiency tradeoffs. The best-performing non-oracle system we could run

¹<https://www.codabench.org/competitions/10361/>

end-to-end is E5 top-10 retrieval plus Qwen3-30B-A3B, with 0.890 M-Prometheus accuracy on development data and 51.09 chrF on the native-text Codabench test subset.

2 Background

CUS-QA. Open-ended regional question answering asks for locally grounded facts, such as the garden associated with a Czech town or the genre of a Slovak music association, that broad multilingual benchmarks fail to capture and current large language models (LLMs) struggle to answer. [Libovický et al. \(2026\)](#) show that even the best open-weight LLMs achieve only around 40% accuracy on textual questions without retrieval, demonstrating that parametric knowledge alone is insufficient for this task. We study how retrieval augmented generation (RAG) can close this gap.

This paper addresses the RAG track of the 10ECTS course IN5550: Neural methods in NLP home examination at the University of Oslo.² The track is based on CUS-QA, a dataset of native-speaker questions grounded in Czech, Slovak, and Wikipedia. Because all dataset knowledge is sourced from Wikipedia, the task is well-suited for RAG experiments.

Retrieval and multilingual embeddings. RAG combines a retriever with a generator so that answers can be grounded in external evidence rather than only in model parameters ([Lewis et al., 2020](#)). This motivates our Oracle/Embedding split: Oracle RAG tests answer extraction when evidence is known, while Embedding RAG tests the realistic retrieval step. We use FineWiki ([Penedo, 2025](#))³, a cleaned Wikipedia-derived corpus, as the retrieval collection and multilingual E5 ([Wang et al., 2024](#)) as the embedding model. E5 is a dense multilingual query-passage embedding model, making it a natural first retriever for Czech, Slovak, and Ukrainian text. It uses different prefixes for queries and passages, so we encode questions with `query:` and FineWiki chunk text with `passage:`. We use FAISS ([Johnson et al., 2019](#)) for nearest-neighbor search because the full FineWiki index contains millions of chunks.

²The track required Multilingual-E5-Large as the embedding baseline, M-Prometheus as the LLM judge, and chrF as the primary lexical metric, but left system architecture and generator choice open.

³<https://huggingface.co/datasets/HuggingFaceFW/finewiki>

Evaluation. Open-ended regional QA is difficult to score by exact match because a correct answer may be a phrase, an inflected form, or a sentence containing the required entity. The task description therefore asks for M-Prometheus as an LLM-as-judge metric on the development set ([Pombal et al., 2025](#)), with chrF ([Popović, 2015](#)) as a secondary lexical metric. These metrics answer different questions: M-Prometheus targets semantic correctness, while chrF measures character-level overlap that is useful for morphologically rich languages. Codabench evaluates hidden test submissions with several automatic metrics and uses chrF as the primary comparison metric for this track.

3 Task and Data

Our experiments use the native textual rows: 1,408 development examples and 1,399 hidden test examples. Development records include the question, reference answer, and source Wikipedia title and URL metadata. This source metadata enables the Oracle RAG condition and target-title retrieval diagnostics. Test records contain only the question metadata, so retrieval and generation must run without answer or source-page supervision. The full CUS-QA⁴ benchmark also includes 548 development and 549 test visual examples, but visual QA and English-translated rows are outside this report.

Lang.	Question	Reference answer	Source title
cs	Jaká známá zahrada se nachází v Bečově nad Teplou?	Bečovská botanická zahrada.	Bečov nad Teplou

Table 1: Example native-text CUS-QA development record. Development examples include the question, reference answer, and source title; hidden-test examples do not expose the answer or source page.

We index only Czech, Slovak, and Ukrainian FineWiki documents. Each question retrieves from FineWiki documents in the same language as the question. This design reduces cross-language noise and keeps the experiment focused on native-text retrieval, but it does not test whether English or neighboring-language evidence could improve coverage. We focus exclusively on the native textual rows; visual questions and English translations are outside the scope of this study. Consequently, our Codabench submissions contain only native-text

⁴<https://huggingface.co/datasets/ufal/cus-qa>

predictions. Appendix Table 7 gives the number of native-language text questions together with development retrieval coverage.

4 System

Figure 1 shows the two RAG settings. Both settings use the same context formatting and generator prompt, but differ in how context is obtained.

Oracle RAG. Oracle RAG uses the source Wikipedia title available in development examples. We first fetch page text with the Wikipedia extracts API. If the extract is empty, we fetch rendered HTML through the Wikipedia parse API and convert it to text with a standard-library HTML parser. This fallback matters for valid pages such as the Czech portal page *Portál:Kroměříž*, where the plain extract can be empty even though the rendered page contains useful text. The final page text is chunked with the same 512-token window and 128-token overlap used by Embedding RAG.

Embedding RAG. For Embedding RAG, FineWiki documents are split into fixed 512-token chunks with 128-token overlap. Full Wikipedia-derived pages are often too broad or noisy to pass directly to the generator, while very short chunks can lose local evidence. The fixed window keeps retrieval units specific, and the overlap preserves context across chunk boundaries; we do not run a chunk-size or structure-aware chunking ablation. Each chunk keeps its title, language, chunk id, document id, URL/source metadata when available, and text. We encode chunk text with `intfloat/multilingual-e5-large`, mean-pool token representations, and L2-normalize embeddings. Since the vectors are normalized, FAISS IndexFlatIP search is equivalent to cosine similarity. We build one index per language and evaluate both top-5 and top-10 retrieval. The stored metadata is used for prompt context headings and retrieval diagnostics.

Generators and decoding. We evaluate CohereLabs/tiny-aya-global, CohereLabs/tiny-aya-water (Salamanca et al., 2026), meta-llama/Llama-3.1-8B-Instruct (Meta AI, 2024), and Qwen/Qwen3-30B-A3B (Qwen Team, 2025). Tiny Aya was chosen because it is a compact 3.35B multilingual model family designed for broad and lower-resourced language coverage. We include Tiny Aya Global as the broad multilingual baseline and Tiny Aya Water

as the same small model family specialized for Europe and Asia-Pacific, making it directly relevant to Czech, Slovak, and Ukrainian. Llama 3.1 8B gives a familiar open-weight instruction baseline, while Qwen3-30B-A3B was the strongest Qwen-family model supported by the stack on Fox, the University of Oslo HPC cluster, and feasible on available hardware. We also considered Gemma 4, newer Qwen3.6 variants, and Llama 70B, but these were excluded from the final comparison because they did not run reliably under the available Fox software and memory constraints. All systems use model chat templates when available and deterministic decoding with temperature 0.0, top- p 1.0, and at most 64 new tokens. Qwen thinking mode is disabled because the task expects concise answers rather than reasoning traces.

5 Evaluation

Retrieval coverage. On development data, retrieved chunk metadata can be compared with the known target Wikipedia title. We report target-title hit rate, or gold-page recall: the proportion of examples where at least one retrieved chunk comes from the source page. We also report target-title MRR, which rewards placing that source-page chunk earlier in the ranking. These are page-level diagnostics, not final QA metrics: they show whether the expected article appears, not whether the answer-bearing passage was selected.

We therefore also compute answer-aware retrieval diagnostics. Strict answer-string recall checks whether the reference answer appears exactly in any retrieved chunk; it is conservative because the evidence may use different wording or inflection. So it works as a lower bound. Relaxed answer-token recall checks whether the main reference-answer tokens appear in the retrieved context; it is useful diagnostically but can overcount short or common answers. A different page may still contain the answer, and a retrieved target page may still lack enough evidence for generation.

Development answer quality. Evaluating open-ended multilingual text requires metrics that capture both semantic correctness and lexical overlap. The primary development metric is M-Prometheus binary correctness. The judge receives the question, reference answer, and model answer, then outputs CORRECT or INCORRECT. An initial test prompt was too strict: it marked a Czech answer containing “Moravské divadlo Olomouc” incorrect against the

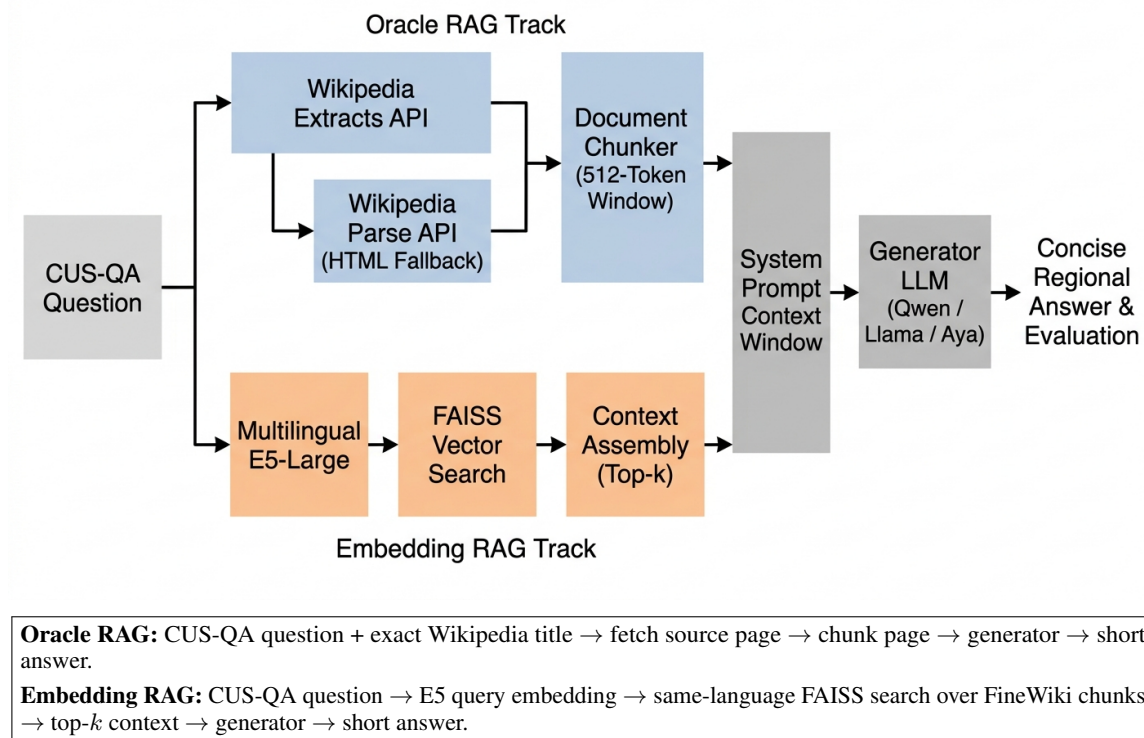


Figure 1: Two evaluation conditions. Oracle RAG isolates generation with exact source-page context, while Embedding RAG is the realistic retrieval condition.

reference “Pro Moravské divadlo Olomouc.” We therefore revised the prompt to accept semantic equivalents, inflection, reordered lists, and harmless extra wording when the required answer is present. In a 9-example smoke set drawn to cover all three languages, the revised prompt corrected the inspected false negative; the full reported rankings use only this final prompt. Appendix A gives the exact template.

We also conducted a 31-example author audit of M-Prometheus judgments, balanced across languages and covering all four generators in Oracle and Embedding RAG settings. For each row, we inspected the question, reference, model answer, retrieved context, judge label, and judge rationale, then assigned a manual label and error note. This audit is a calibration and error-analysis check, not a full native-speaker or multi-annotator evaluation.

M-Prometheus agreed with the manual label on 28 / 30 comparable examples, or 28 / 31 if the uncertain ambiguous-entity case is counted as non-agreement. The clear disagreements involved reference specificity, where concise answers captured the main fact but omitted details in the reference. The audit therefore supports M-Prometheus as a useful automatic semantic judge while showing that

it is not human ground truth. The audit also showed that some failures had the correct page or answer evidence in context, but generation followed a distractor, selected the wrong attribute, or struggled with flattened table structure. Appendix Table 12 summarizes these categories.

We also compute chrF, a character n-gram F-score. chrF is useful for morphologically rich languages and hidden-test comparison because it can reward partial overlap between inflected forms, but it remains lexical and is not sufficient for semantic model selection on its own.

Hidden test evaluation. For the hidden test set, Codabench reports BLEURT, BLEU, chrF, ROUGE-L, and BERTScore. We use chrF as the primary hidden-test metric, following the task instructions. Since we only submit native text, final Codabench scores over all modalities are not populated; we report the regional native-text results and weighted native-text averages. Table 3 and Figure 5 report these native-text Codabench results.

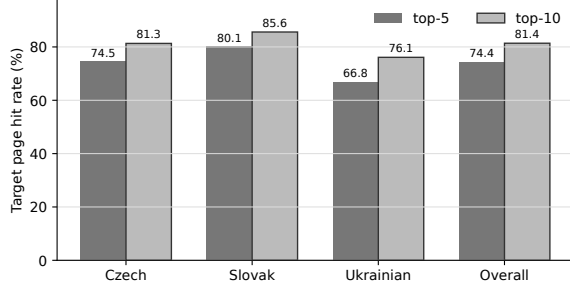


Figure 2: Exact target-title hit rate for E5 retrieval. Top-10 improves coverage for all languages, but Ukrainian remains the hardest language.

6 Results

6.1 Retrieval Coverage

Appendix Table 7 and Figure 2 show that E5 retrieval is a major bottleneck. Top-5 retrieval finds the target page for 74.4% of development questions, while top-10 raises this to 81.4%, recovering 99 additional target pages. The gain is consistent across languages, but Ukrainian remains lowest at 66.8% for top-5 and 76.1% for top-10.

The answer-aware diagnostics in Appendix Figure 7 show why more retrieved chunks do not automatically yield better answers. Gold-page recall rises from 0.7436 to 0.8139, but MRR barely changes from 0.5867 to 0.5962, meaning the additional source-page chunks are often added lower in the ranking rather than promoted near the top. Strict answer-string recall also improves only modestly, from 0.6357 to 0.6690. Thus top-10 improves coverage, but it also leaves evidence selection and ranking quality unresolved. This foreshadows the answer-quality results below: stronger models can sometimes use the extra context, while smaller models are more easily distracted by it.

6.2 Generator Quality

Table 2 gives the main development ranking. With known-source chunks, Qwen3-30B reaches 0.953 M-Prometheus accuracy and Llama-8B reaches 0.934, showing that the generators can answer many questions when the source page is supplied. In the realistic Embedding RAG setting, Qwen3-30B with E5 top-10 is strongest at 0.890. It outperforms all other retrieved-context systems and nearly matches the Oracle Tiny Aya models, suggesting that generator strength can partly compensate for imperfect retrieval. The Oracle setting is still chunk-limited: it is an upper bound for our RAG pipeline, not for a model given the full source

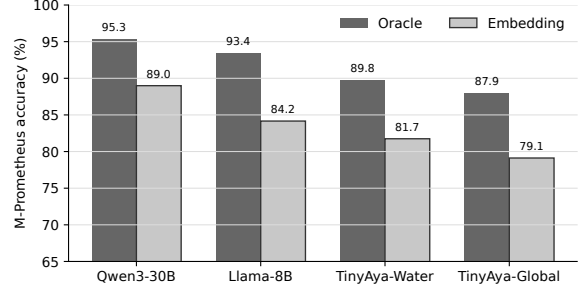


Figure 3: Oracle-to-Embedding gap using overall development M-Prometheus accuracy across languages. Even the strongest Embedding RAG system remains below its Oracle counterpart.

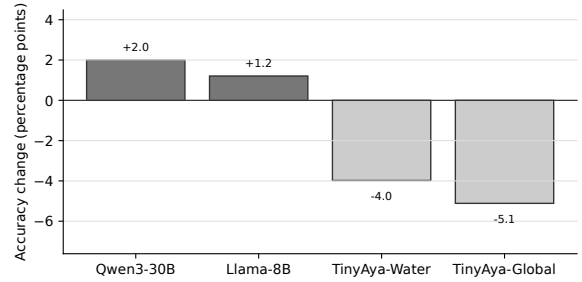


Figure 4: M-Prometheus change from top-5 to top-10 retrieval. More context helps Qwen3-30B and Llama-8B, but hurts both Tiny Aya variants.

article. Appendix Table 9 and Appendix Figure 6 provide the full by-language M-Prometheus results.

Figure 3 makes the context-quality gap explicit. Holding the generator fixed, Oracle systems consistently outperform Embedding RAG systems. For Qwen3-30B, accuracy drops from 0.953 with known-source chunks to 0.890 with top-10 retrieved FineWiki chunks. Since decoding, prompt format, and answer-length settings are shared across conditions, the gap is best explained by context differences: missing source pages, answer evidence ranked too low, and distracting retrieved passages.

6.3 Top- k Tradeoff

Increasing k is not uniformly beneficial. Figure 4 shows that top-10 improves Qwen3-30B by 2.0 points and Llama-8B by 1.2 points. The same change hurts Tiny Aya Water by 4.0 points and Tiny Aya Global by 5.1 points. Appendix Table 7 provides the exact numerical values. This pattern is consistent with a coverage-versus-distraction tradeoff: top-10 retrieves the target page more often, but it also adds irrelevant or weakly related evidence. However, our experiments do not isolate the mech-

System	Condition	k	Generator	Accuracy	Correct
Oracle Qwen3-30B	Oracle	–	Qwen3-30B	0.9531	1342 / 1408
Oracle Llama-8B	Oracle	–	Llama-8B	0.9339	1315 / 1408
Oracle TinyAya-Water	Oracle	–	TinyAya-Water	0.8977	1264 / 1408
Qwen3-30B top-10	Embedding	10	Qwen3-30B	0.8899	1253 / 1408
Oracle TinyAya-Global	Oracle	–	TinyAya-Global	0.8793	1238 / 1408
Qwen3-30B top-5	Embedding	5	Qwen3-30B	0.8700	1225 / 1408
Llama-8B top-10	Embedding	10	Llama-8B	0.8416	1185 / 1408
Llama-8B top-5	Embedding	5	Llama-8B	0.8295	1168 / 1408
TinyAya-Water top-5	Embedding	5	TinyAya-Water	0.8175	1151 / 1408
TinyAya-Global top-5	Embedding	5	TinyAya-Global	0.7912	1114 / 1408
TinyAya-Water top-10	Embedding	10	TinyAya-Water	0.7777	1095 / 1408
TinyAya-Global top-10	Embedding	10	TinyAya-Global	0.7401	1042 / 1408

Table 2: Main development results with M-Prometheus. Oracle systems use the exact Wikipedia source page but still operate through the same chunked context pipeline.

anism. Shorter chunks, reranking, or controlled distractor experiments would be needed to determine whether the dominant factor is context length, evidence selection, or instruction-following robustness. Between the two Tiny Aya variants, Water is stronger on development M-Prometheus at top-5, while Global is slightly stronger on hidden-test native-text chrF; both show the same degradation under top-10 retrieval.

6.4 Reranking Pilot

As a final, time-constrained experiment, we tested whether the Tiny Aya Water drop is partly caused by noisy evidence ordering. A multilingual BGE cross-encoder reranker (Li et al., 2023; Chen et al., 2024) reorders the E5 top-10 candidates, after which Tiny Aya Water receives either the reranked top-5 or reranked top-10 chunks. We restrict this pilot to Tiny Aya Water because shared Fox queue time and compute limits prevented a full reranking sweep, and because Tiny Aya Water is the strongest practical low-resource candidate in our runs: it completes top-5 generation in about 19 minutes, uses low memory, and remains close enough to Llama to be interesting for deployment-oriented settings.

Reranking improves Tiny Aya Water in both final-context settings. Raw E5 top-5 gives 0.8175 M-Prometheus accuracy and 37.10 chrF, while reranked top-5 reaches 0.8450 and 38.56. Raw E5 top-10 gives 0.7777 M-Prometheus accuracy and 36.56 chrF, while reranked top-10 reaches 0.8053 and 39.23. Thus reranking recovers much of the top-10 degradation and improves over the original top-5 setting, but reranked top-5 remains best for Tiny Aya Water. Cleaner evidence helps, yet this small model still benefits from a shorter final

System	k	chrF	BLEURT	BLEU	BERT
TinyAya-Water	5	37.72	0.453	4.64	0.539
TinyAya-Global	5	38.22	0.467	4.88	0.560
Llama-8B	10	41.27	0.468	6.10	0.535
Qwen3-30B	10	51.09	0.559	10.94	0.635

Table 3: Codabench native-text partial test results by submitted Embedding RAG configuration. Scores are weighted by native-text test counts. chrF is the primary hidden-test metric for this track.

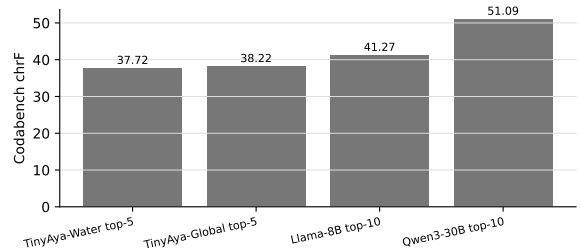


Figure 5: Weighted native-text Codabench chrF. Qwen3-30B top-10 transfers best from development to hidden test.

context. Appendix Figure 8 visualizes the pilot results.

6.5 Hidden Test Results

Table 3 reports the native-text partial Codabench submissions that correspond to the main Embedding RAG configurations. The best hidden-test row is Qwen3-30B with E5 top-10 retrieval.

The hidden-test results agree with the development ranking. Qwen3-30B top-10 obtains 51.09 weighted native-text chrF, far above Llama-8B top-10 at 41.27 and the Tiny Aya systems near 38. Qwen is also strongest on BLEURT, BLEU, and BERTScore. Region-level Codabench results show the same difficulty pattern as retrieval: Qwen

reaches 54.26 chrF for Czech and 51.80 for Slovak, but 45.46 for Ukrainian.

6.6 Quality-Efficiency Tradeoff

Runtime and memory observations were collected on Fox, the UiO HPC cluster. They are practical run observations rather than a controlled latency benchmark.

Table 4 makes the model-selection tradeoff explicit. We compute marginal cost as $(\text{runtime}_B - \text{runtime}_A)/(\text{correct}_B - \text{correct}_A)$, reported as extra runtime minutes per additional correct answer. Tiny Aya Water top-5 is a strong low-resource option, running much faster and using less memory while producing results roughly 2.4 M-Prometheus points below Llama top-10. Llama-8B top-10 serves as a familiar moderate-cost comparison, adding 34 correct development answers. Qwen3-30B top-10 adds another 68 correct answers at a similar observed marginal runtime cost, but requires much more GPU memory and an A100-class setting. We therefore treat Qwen as the final-quality choice, Llama as a moderate-cost comparison, and Tiny Aya as a strong low-resource option.

7 Discussion

Our experiments highlight multiple takeaways regarding retrieval importance and model tradeoffs under constrained settings.

Retrieval remains the bottleneck. The Oracle-to-Embedding gap is the clearest result. Oracle RAG uses the known source page, but the generator still receives selected chunks rather than the full article, so it is an upper bound for our chunked RAG pipeline. Embedding RAG is the realistic setting and remains lower for every generator. The gap is especially meaningful for Qwen3-30B: a strong generator still loses 6.3 absolute accuracy points when known-source chunks are replaced by top-10 retrieved FineWiki chunks. The retrieval diagnostics explain why: top-10 improves gold-page recall, but target-title MRR barely changes, so additional source-page evidence is often added lower in the ranking rather than promoted near the top. Ukrainian is the clearest stress case, with the lowest retrieval coverage and hidden-test chrF.

More context is not always better. Top-10 retrieval improves target-title coverage for all languages, but only the stronger generators convert that extra coverage into better answers. Qwen3-30B and Llama-8B can use the additional evidence,

while Tiny Aya models are more sensitive to distractors. The author audit supports this interpretation: several failures involved correct pages or answer-bearing evidence in context, but generation followed a higher-ranked distractor, selected the wrong attribute, or struggled with flattened table structure. In practice, k should therefore be tuned jointly with the generator rather than selected from retrieval coverage alone.

Reranking helps the small-model setting. The reranking pilot supports the evidence-ordering interpretation. Reordering E5 top-10 candidates with a multilingual BGE reranker improves Tiny Aya Water in both final-context settings: reranked top-5 is stronger than raw top-5, and reranked top-10 is stronger than raw top-10. However, reranked top-5 remains the best Tiny Aya Water setting, which suggests that cleaner candidate selection helps but the small model still benefits from a concise final context. This is a focused pilot rather than a complete reranker evaluation: only Tiny Aya Water was tested, only E5 top-10 candidates were reranked, and no hidden-test submission was made from reranked outputs.

Quality and efficiency differ by deployment point. The best model depends on the operating point. For low-resource experiments, Tiny Aya Water top-5 is attractive because it finishes quickly and uses little memory. For a moderate-cost comparison, Llama-8B top-10 is stronger. For final quality, Qwen3-30B top-10 is clearly best, but it requires substantially more memory and longer runtime. We do not introduce a formal latency-adjusted metric because the jobs ran under practical cluster conditions rather than a controlled hardware benchmark, but Table 4 gives a transparent quality-efficiency view.

Metrics answer different questions. chrF and M-Prometheus agree that Qwen3-30B is the strongest system, but they can reorder middle-ranked systems. For example, Embedding Qwen can score higher by chrF than Oracle Llama-8B, while Oracle Llama scores higher by M-Prometheus. This is expected: chrF rewards character overlap with the reference, while M-Prometheus checks whether the answer is semantically correct. At the same time, M-Prometheus remains an automatic judge rather than human evaluation. We therefore use M-Prometheus as the primary development metric and chrF as a secondary diagnostic

System	Setting	Acc.	Correct	Runtime	Peak GPU Mem	+Correct	min/+corr.
TinyAya-Water	top-5	0.8175	1151 / 1408	0:19:29	7.7 GiB / 1 GPU	–	–
TinyAya-Global	top-5	0.7912	1114 / 1408	0:19:42	7.7 GiB / 1 GPU	-37	–
Llama-8B	top-10	0.8416	1185 / 1408	1:11:56	16.4 GiB / 1 GPU	+34	1.54
Qwen3-30B	top-10	0.8899	1253 / 1408	2:54:08	60.7 GiB / A100	+68	1.50

Table 4: Practical quality-efficiency comparison for four useful Embedding RAG operating points. Runtime and memory are observed on Fox, the UiO HPC cluster, not controlled latency benchmarks; one-GPU rows likely ran on RTX 3090-class GPUs, while Qwen used an A100-class setting. Additional correct answers and minutes per additional correct answer are computed relative to the previous row.

and hidden-test comparison.

8 Limitations

This study covers only native-text Czech, Slovak, and Ukrainian questions. We do not handle visual questions or English-translated rows, so Codabench submissions are partial. We use only multilingual E5 as the base retriever and do not test hybrid sparse+dense retrieval. Due to shared HPC queue time and compute limits, we ran reranking only for Tiny Aya Water, not for Qwen, Llama, or larger candidate pools. The model comparison is also limited to systems that ran reliably on the available Fox software stack and GPU memory; Gemma, newer Qwen variants, and Llama 70B were considered but excluded from final runs. M-Prometheus is an automatic judge rather than human evaluation, and its behavior depends on prompt design. The 64-token generation cap encourages concise answers but can truncate occasional list-style answers. Finally, FineWiki and current Wikipedia API responses may not exactly match the Wikipedia snapshots used when CUS-QA was authored, which can create apparent retrieval or Oracle-context failures even when the title is correct.

9 Conclusion

We compared Oracle and Embedding RAG for native-text regional QA in Czech, Slovak, and Ukrainian. Qwen3-30B-A3B is the strongest generator in both settings, and E5 top-10 plus Qwen3-30B is the best realistic system among our runs. However, the gap between Oracle and Embedding RAG shows that retrieval quality remains a central limitation. Increasing retrieval depth helps stronger models but distracts smaller models, making the optimal amount of context dependent on generator capacity. For this task, the best practical recipe is not simply “retrieve more”, but retrieve enough evidence for a generator that can use it. The Tiny Aya Water reranking pilot supports this interpreta-

tion: cleaner candidate ordering improves the small model, but its best setting still uses a concise final context. Future work should extend reranking and hybrid retrieval across generators, while answer-level diagnostics and table-aware chunking would help explain the remaining errors.

Ethics and AI Assistance

This work utilizes public benchmark data released exclusively for research and academic evaluation. The experiments are restricted to open-weight language models and involve no human subjects beyond the original creators of the CUS-QA dataset. Large language models were utilized for code generation, data analysis, and text drafting; however, all experimental designs, model selections, and final interpretations of the empirical results were conducted entirely by the human authors.

References

- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. [BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation](#). *Preprint*, arXiv:2402.03216.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.
- Chaofan Li, Zheng Liu, Shitao Xiao, and Yingxia Shao. 2023. [Making large language models a better foundation for dense retrieval](#). *Preprint*, arXiv:2312.15503.
- Jindřich Libovický, Jindřich Helcl, Andrei-Alexandru Manea, and Gianluca Vico. 2026. [CUS-QA: Local-](#)

knowledge-oriented open-ended question answering dataset. *Preprint*, arXiv:2507.22752.

Meta AI. 2024. [Llama 3.1 8b instruct model card](#).

Guilherme Penedo. 2025. [FineWiki](#). Source: Wikimedia Enterprise Snapshot API; text licensed under CC BY-SA 4.0 with attribution to Wikipedia contributors.

José Pombal, Dongkeun Yoon, Patrick Fernandes, Ian Wu, Seungone Kim, Ricardo Rei, Graham Neubig, and André Martins. 2025. [M-Prometheus: A suite of open multilingual LLM judges](#). In *Second Conference on Language Modeling*.

Maja Popović. 2015. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal. Association for Computational Linguistics.

Qwen Team. 2025. [Qwen3 technical report](#). *Preprint*, arXiv:2505.09388.

Alejandro R. Salamanca, Diana Abagyan, Daniel D’souza, Ammar Khairi, David Mora, Saurabh Dash, Viraat Aryabumi, Sara Rajae, Mehrnaz Mofakhami, Ananya Sahu, Thomas Euyang, Brittawnya Prince, Madeline Smith, Hangyu Lin, Acyr Locatelli, Sara Hooker, Tom Kocmi, Aidan Gomez, Ivan Zhang, and 7 others. 2026. [Tiny Aya: Bridging scale and multilingual depth](#). *Preprint*, arXiv:2603.11510.

Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024. [Multilingual E5 text embeddings: A technical report](#). *Preprint*, arXiv:2402.05672.

A Prompt Templates

A.1 Generator Prompt

The same generator prompt template is used for Oracle and Embedding RAG. The language instruction forces answers to stay in the question language, the context-only constraint discourages unsupported parametric answers, and the fallback phrase allows a short answer when evidence is incomplete rather than forcing an empty output. The context block is filled with either chunks from the exact source page or retrieved FineWiki chunks.

Answer the question in {language_name}. Use only the context below. If the context is insufficient, give the best short answer supported by it.

Question:
{question}

Context:
[{rank}] {title}
{text}

Short answer:

A.2 M-Prometheus Judge Prompt

The final judge prompt version is v2_semantic_harmless_extra. It was introduced after a smoke run showed that a stricter prompt could reject a correct answer with harmless extra wording.

###Task Description:

You are judging an open-ended factual question answering response. Given a question, a reference answer, and a model answer, decide whether the model answer is semantically correct.

Accept equivalent wording, inflection, spelling variants, reordered lists, and harmless extra context if it does not contradict the reference.

If the model answer contains the required entity, place, person, date, number, or list from the reference, mark it CORRECT even if it also states the relation or activity type.

Do not penalize answers for adding relevant words such as "music", "theatre", "city", "district", or "metro" when the requested factual answer is still present.

Mark the answer INCORRECT only if it omits a required fact, adds a contradictory fact, answers a different question, or is too vague to verify.

First write brief feedback. Then write exactly one final label in this format:
[RESULT] CORRECT
or
[RESULT] INCORRECT

###Question:
{question}

###Reference Answer:
{reference}

###Model Answer:
{answer}

###Score Rubric:
CORRECT: The model answer contains the same factual answer as the reference, allowing paraphrases and harmless extra detail.
INCORRECT: The model answer is wrong, incomplete, unsupported, contradictory, or does not answer the question.

###Feedback:

B Experimental Setup and Constraints

Table 5 summarizes the fixed and varied experimental settings. Table 6 records the main track and infrastructure constraints that shaped the final system selection.

C Full Development Tables

This appendix section gives complete development-set scores. Tables 8 and 9 include all completed Oracle and Embedding RAG systems, Figure 6 visualizes the M-Prometheus scores, and Table 7 provides supplementary numeric sources for retrieval coverage and top- k tradeoffs.

D Retrieval Evidence Diagnostics

Figure 7 refines the target-title analysis. It compares page-level gold-source retrieval with simple answer-string proxies, but it should not be read as semantic evidence recall: strict answer-string matching is a lower bound, while relaxed token matching can overcount short or common answers.

E Reranking Pilot Diagnostics

Figure 8 summarizes the focused Tiny Aya Water reranking pilot. The reranker is applied only as a candidate-selection layer over E5 top-10 retrieval; generation, decoding, and evaluation remain otherwise unchanged.

F Metric Disagreement

This appendix section supports the discussion of lexical versus semantic evaluation. Table 10 gives exact chrF and M-Prometheus ranks, while Figure 9 visualizes the same disagreement.

G Model, Runtime, and Resource Summary

Table 11 documents practical resource constraints. Completed rows are observed development runs; blocked rows record model families that were considered but could not be made reliable under the available infrastructure.

These failed stretch runs were not included in the central comparison because they were infrastructure limitations rather than completed systems. They are still useful for interpretation: practical

reproducibility on shared HPC infrastructure constrained the model set, so the final comparison emphasizes systems that completed reliably.

H Qualitative Error and Diagnostic Notes

This appendix section lists concrete diagnostic patterns that motivated the discussion. The examples are not a full error taxonomy, but they make the aggregate claims easier to inspect.

Rendered Wikipedia fallback. The Czech page *Portál:Kroměříž* is a valid Wikipedia source page, but the normal extract endpoint can return empty text for portal pages. The rendered-page fallback recovers useful context and prevents Oracle RAG from silently using an empty source page.

Top-10 distraction. Top-10 retrieval increases exact-title coverage, but Tiny Aya Global and Water both lose accuracy with top-10 compared with top-5. This supports the interpretation that smaller generators are more sensitive to additional irrelevant chunks.

Page-level retrieval optimism. The manual audit showed that retrieval failures are not only missing-page failures. In several cases the correct page appeared in the retrieved set, and sometimes the answer-bearing evidence was present, but generation followed a higher-ranked distractor or selected the wrong attribute. Table-format artifacts in plain-text chunks also caused errors, supporting reranking and structure-aware chunking as practical next steps.

The categories in Table 12 explain why reranking is a plausible next intervention. Several failures are ranking or salience failures rather than total retrieval misses: the correct page or answer-bearing passage is present, but a competing chunk is easier for the generator to follow. A reranker could promote the question-relevant chunk before generation, while table-aware chunking is needed for failures caused by flattened Wikipedia tables. Other categories, such as source ambiguity and reference-specific judge disagreements, show that retrieval alone cannot solve every error.

Judge specificity. The audit also revealed a specificity mismatch. Our generator prompt asks for concise answers, while some CUS-QA references provide a direct answer plus additional explanatory detail. In such cases, M-Prometheus can mark a

Component	Setting	Role
Data subset	Native-text Czech, Slovak, and Ukrainian CUS-QA rows	Fixed
Retrieval corpus	Same-language Czech, Slovak, and Ukrainian FineWiki	Fixed
Retriever	intfloat/multilingual-e5-large with query/passage prefixes	Fixed
Index	FAISS IndexFlatIP over L2-normalized embeddings	Fixed
Chunking	512-token windows with 128-token overlap	Fixed
Context depth	top-5 or top-10 retrieved chunks	Varied
Generator	Tiny Aya Global, Tiny Aya Water, Llama-8B, Qwen3-30B	Varied
Decoding	temperature 0.0, top- p 1.0, max 64 new tokens	Fixed
Development metric	M-Prometheus accuracy, with chrF as secondary	Fixed
Hidden-test metric	Codabench native-text chrF and auxiliary metrics	Fixed

Table 5: Summary of deterministic and varied experimental settings.

Constraint	Effect in this report	If unconstrained
Native-text scope	We evaluate only Czech, Slovak, and Ukrainian textual rows; visual and English-translated rows are outside scope.	Add visual QA and English-translated rows for a complete Codabench submission.
Same-language retrieval	Each question searches only the matching FineWiki language, reducing cross-language noise.	Test shared multilingual indexes or query/document translation.
Single dense retriever	Multilingual E5 is the only base retriever in the main runs.	Compare BGE-M3, LaBSE, BM25, and hybrid sparse+dense retrieval.
Limited reranking pilot	Reranking is tested only for Tiny Aya Water over E5 top-10 candidates because of queue time and compute limits.	Rerank top-10/top-20 candidates for all generators and hidden-test submissions.
Fixed chunking	We use one 512/128 fixed-window setting.	Run chunk-size and section/table-aware chunking ablations.
Shared HPC stack	Final systems are limited to models that ran reliably on available Fox modules and GPUs.	Evaluate larger/newer generators and more backends under stable software support.

Table 6: Track, retrieval, and compute constraints, with natural extensions that would be tested without those constraints.

Retrieval coverage					Top- k ablation			
Language	Dev	Test	top-5 hit	top-10 hit	Generator	top-5	top-10	Delta
Czech	530	550	74.5	81.3	Qwen3-30B	0.8700	0.8899	+2.0 pp
Slovak	493	479	80.1	85.6	Llama-8B	0.8295	0.8416	+1.2 pp
Ukrainian	385	370	66.8	76.1	TinyAya-Water	0.8175	0.7777	-4.0 pp
Overall	1408	1399	74.4	81.4	TinyAya-Global	0.7912	0.7401	-5.1 pp

Table 7: Supplementary development retrieval and top- k diagnostics. Left: native-text development and test sizes with development target-title retrieval coverage. Right: effect of increasing retrieved context from top-5 to top-10, measured by development M-Prometheus accuracy.

System	Overall	cs	sk	uk
Oracle Qwen3-30B	56.5152	59.6681	57.9848	52.7789
Qwen3-30B top-5	49.9479	53.0567	52.5310	45.3271
Qwen3-30B top-10	49.4242	52.4260	50.8699	45.7527
Oracle Llama-8B	45.8891	46.3905	46.2559	45.1969
Llama-8B top-5	40.5242	40.2660	41.7506	39.8043
Llama-8B top-10	40.5079	40.0808	41.5056	40.1463
Oracle TinyAya-Water	39.8922	40.4136	42.1370	37.7748
Oracle TinyAya-Global	39.8747	41.1366	41.7626	37.3524
TinyAya-Global top-5	37.2938	38.3314	38.5950	35.4144
TinyAya-Water top-5	37.1012	37.2283	38.5854	35.8894
TinyAya-Global top-10	37.0715	39.2065	38.3515	34.2275
TinyAya-Water top-10	36.5558	37.7640	37.4017	34.8699

Table 8: Full chrF development results for all evaluated systems.

System	Overall	cs	sk	uk	Correct
Oracle Qwen3-30B	0.9531	0.9604	0.9594	0.9351	1342 / 1408
Oracle Llama-8B	0.9339	0.9472	0.9452	0.9013	1315 / 1408
Oracle TinyAya-Water	0.8977	0.9113	0.9351	0.8312	1264 / 1408
Qwen3-30B top-10	0.8899	0.9000	0.9148	0.8442	1253 / 1408
Oracle TinyAya-Global	0.8793	0.9000	0.9148	0.8052	1238 / 1408
Qwen3-30B top-5	0.8700	0.8755	0.9026	0.8208	1225 / 1408
Llama-8B top-10	0.8416	0.8679	0.8763	0.7610	1185 / 1408
Llama-8B top-5	0.8295	0.8623	0.8600	0.7455	1168 / 1408
TinyAya-Water top-5	0.8175	0.8226	0.8621	0.7532	1151 / 1408
TinyAya-Global top-5	0.7912	0.8094	0.8276	0.7195	1114 / 1408
TinyAya-Water top-10	0.7777	0.8113	0.7830	0.7247	1095 / 1408
TinyAya-Global top-10	0.7401	0.7981	0.7404	0.6597	1042 / 1408

Table 9: Full M-Prometheus development accuracy by system and language.

System	Condition	k	Generator	chrF	M-Prom.	chrF rank	Judge rank
Oracle Qwen3-30B	Oracle	-	Qwen3-30B	56.5152	0.9531	1	1
Oracle Llama-8B	Oracle	-	Llama-8B	45.8891	0.9339	4	2
Oracle TinyAya-Water	Oracle	-	TinyAya-Water	39.8922	0.8977	7	3
Qwen3-30B top-10	Embedding	10	Qwen3-30B	49.4242	0.8899	3	4
Oracle TinyAya-Global	Oracle	-	TinyAya-Global	39.8747	0.8793	8	5
Qwen3-30B top-5	Embedding	5	Qwen3-30B	49.9479	0.8700	2	6
Llama-8B top-10	Embedding	10	Llama-8B	40.5079	0.8416	6	7
Llama-8B top-5	Embedding	5	Llama-8B	40.5242	0.8295	5	8
TinyAya-Water top-5	Embedding	5	TinyAya-Water	37.1012	0.8175	10	9
TinyAya-Global top-5	Embedding	5	TinyAya-Global	37.2938	0.7912	9	10
TinyAya-Water top-10	Embedding	10	TinyAya-Water	36.5558	0.7777	12	11
TinyAya-Global top-10	Embedding	10	TinyAya-Global	37.0715	0.7401	11	12

Table 10: Exact chrF and M-Prometheus scores. The differing ranks show why chrF is useful but not sufficient for development-set model selection.

Model / family	Status	Backend	Resource note (peak GPU RAM)	Runtime (hh:mm) / outcome
Tiny Aya Global	completed	Transformers	1 GPU, 7.7 GiB	0:18 top-5, 1:01 top-10
Tiny Aya Water	completed	Transformers	1 GPU, 7.7 GiB	0:19 top-5, 1:06 top-10
Llama 3.1 8B	completed	Transformers	1 GPU, 16.4 GiB	0:47 top-5, 1:12 top-10
Qwen3-30B-A3B	completed	Transformers	A100-class, 60–64 GiB	2:34 top-5, 2:54 top-10
Gemma 4	blocked	Transformers/vLLM	module compatibility	model load/runtime failure
Qwen3.6	blocked	vLLM	module version mismatch	failed before usable run
Llama 70B AWQ	blocked	vLLM	2 RTX30 too tight	KV-cache/profiling OOM

Table 11: Model runtime and resource summary. GiB indicates peak GPU memory (RAM), and runtimes are formatted as hours:minutes. Completed runtimes are full-development Embedding RAG observations; blocked models are listed to document practical reproducibility constraints.

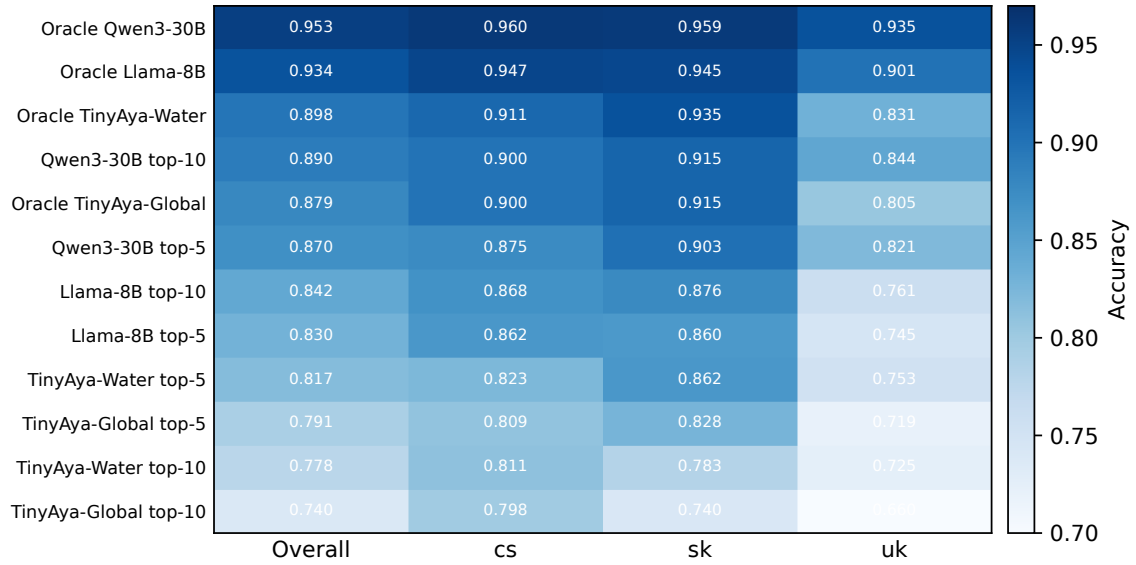


Figure 6: M-Prometheus accuracy by system and language.

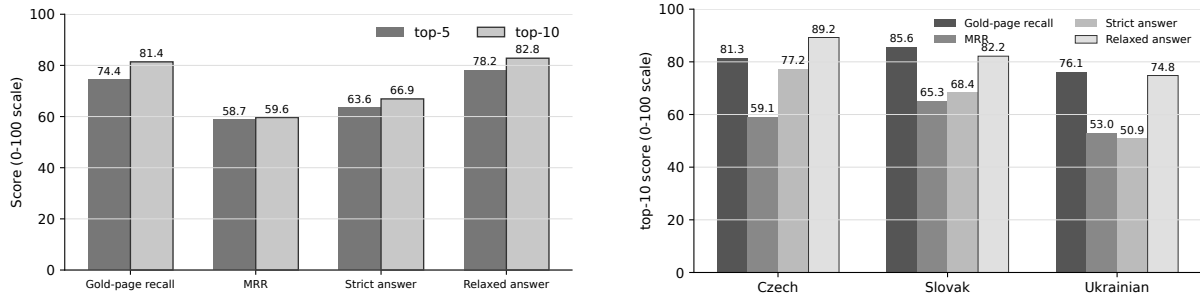


Figure 7: Retrieval evidence diagnostics by retrieval depth and language. Gold-page recall is the same page-level signal used for target-title hit rate; strict answer-string recall is a conservative proxy, and relaxed answer-token recall is diagnostic rather than proof that answer-bearing evidence was semantically retrieved.

Audit pattern	CUS-QA dev IDs
Clear retrieval miss	328, 374
Correct page / distractor	313 (2 rows), 260, 173
Evidence present, not extracted	265, 65
Oracle chunk limitation	306, 90
Source/entity ambiguity	294, 28, 446
Judge/reference specificity	477, 362
Table-structure failure	102
Generation length issue	357

Table 12: Qualitative audit patterns mapped to original CUS-QA development example IDs. The prose below explains the interpretation of each category.

concise answer incorrect because it omits reference-specific detail, even when it answers the main question. This makes judge strictness a real design choice: correctness can mean either answering the main question or matching the full specificity of the reference.

Ukrainian difficulty. Ukrainian has the lowest target-title hit rate and the lowest Qwen hidden-test

chrF. Some development audit cases also showed source-version or wording differences, such as answers that appear with different morphology or script variants.

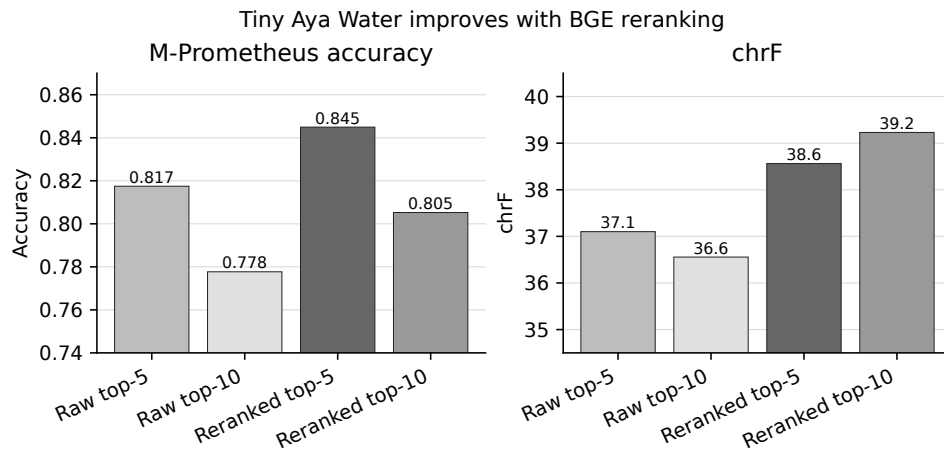


Figure 8: Tiny Aya Water raw E5 versus BGE-reranked retrieval. Reranking reorders E5 top-10 candidates before generation.

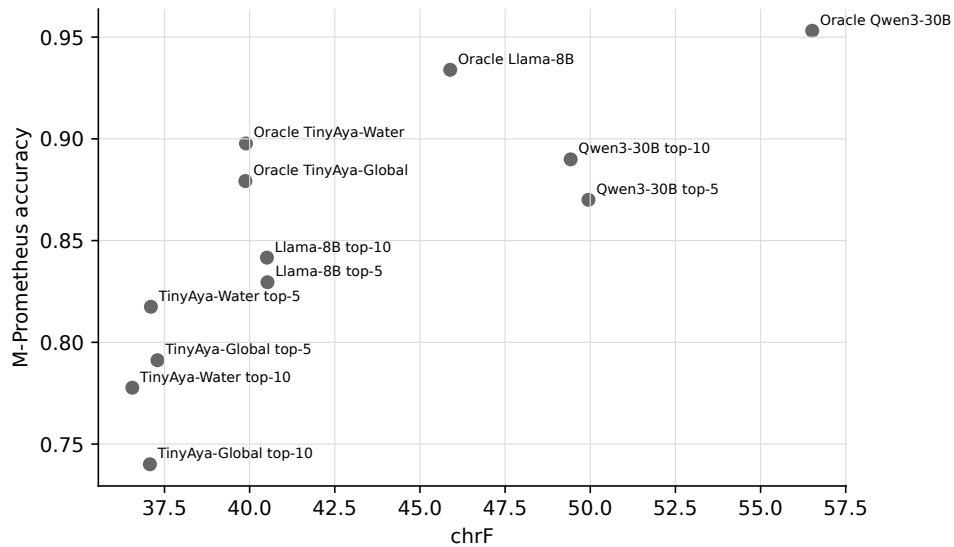


Figure 9: chrF versus M-Prometheus accuracy. Systems with similar lexical overlap can differ in semantic correctness.